

SEDS 537 – Machine Learning

Take-Home Midterm Examination

Instructor: Prof. Dr. Aytug Onan

Midterm Report

Five Machine Learning Tasks

Regression · Classification · Dimensionality Reduction

Clustering · Neural Networks

Student ID: 323011020

Date: April 2026

Contents

Introduction	2
1 Question 1: Regression – Predicting House Prices	3
1.1 Objective and Dataset	3
1.2 Exploratory Data Analysis	3
1.3 Models and Evaluation	4
1.4 Impact of Scaling	5
1.5 Polynomial Features	5
1.6 Residual Plots	5
1.7 Conclusion: Interpretability vs Predictive Performance	6
2 Question 2: Classification – Handling Class Imbalance	7
2.1 Dataset and Class Distribution	7
2.2 Preprocessing	7
2.3 Classifiers	7
2.4 Resampling Techniques	7
2.5 Results	8
2.6 Confusion Matrices and ROC Curves	8
2.7 Precision-Recall Trade-off	9
2.8 Conclusion	9
3 Question 3: Dimensionality Reduction and Visualisation	10
3.1 Dataset	10
3.2 PCA: 2D and 50D	10
3.3 t-SNE: 2D Embeddings	10
3.4 k-NN Classification Comparison	11
3.5 PCA vs t-SNE Discussion	11
4 Question 4: Clustering – Unsupervised Learning	12
4.1 Dataset	12
4.2 K-Means: Choosing k	12
4.3 Agglomerative Clustering (Ward Linkage)	12
4.4 DBSCAN: Tuning eps	13
4.5 Evaluation Metrics	13
4.6 Cluster Visualisation	13
4.7 Cluster Interpretation	14
4.8 DBSCAN Sensitivity	14
4.9 Algorithm Comparison	14
5 Question 5: Neural Networks – MLP vs CNN	15
5.1 Dataset	15
5.2 Models	15
5.3 Training Curves	16
5.4 Test Set Performance	16
5.5 Confusion Matrices	17
5.6 Misclassified Examples	17
5.7 Why CNN Performs Better	18
6 Overall Conclusion	19

Introduction

This report presents the solutions for the five questions of the SEDS 537 Machine Learning take-home midterm. Each question covers a different topic in machine learning:

- **Question 1:** Regression on California Housing data.
- **Question 2:** Classification with class imbalance (Credit Card Fraud).
- **Question 3:** Dimensionality reduction with PCA and t-SNE on MNIST.
- **Question 4:** Clustering on the Wholesale Customer dataset.
- **Question 5:** Neural networks (MLP vs CNN) on Fashion-MNIST.

All experiments are reproducible. We used `random_state=42` everywhere. The code is provided as Jupyter notebooks (Q1–Q5), and all figures shown in this report are generated from those notebooks.

1 Question 1: Regression – Predicting House Prices

1.1 Objective and Dataset

The goal is to compare regression models on the **California Housing** dataset. The dataset has 20,640 samples and 8 numerical features. The target is the median house value (in \$100,000s).

1.2 Exploratory Data Analysis

We first looked at feature distributions, correlations, and scatter plots.

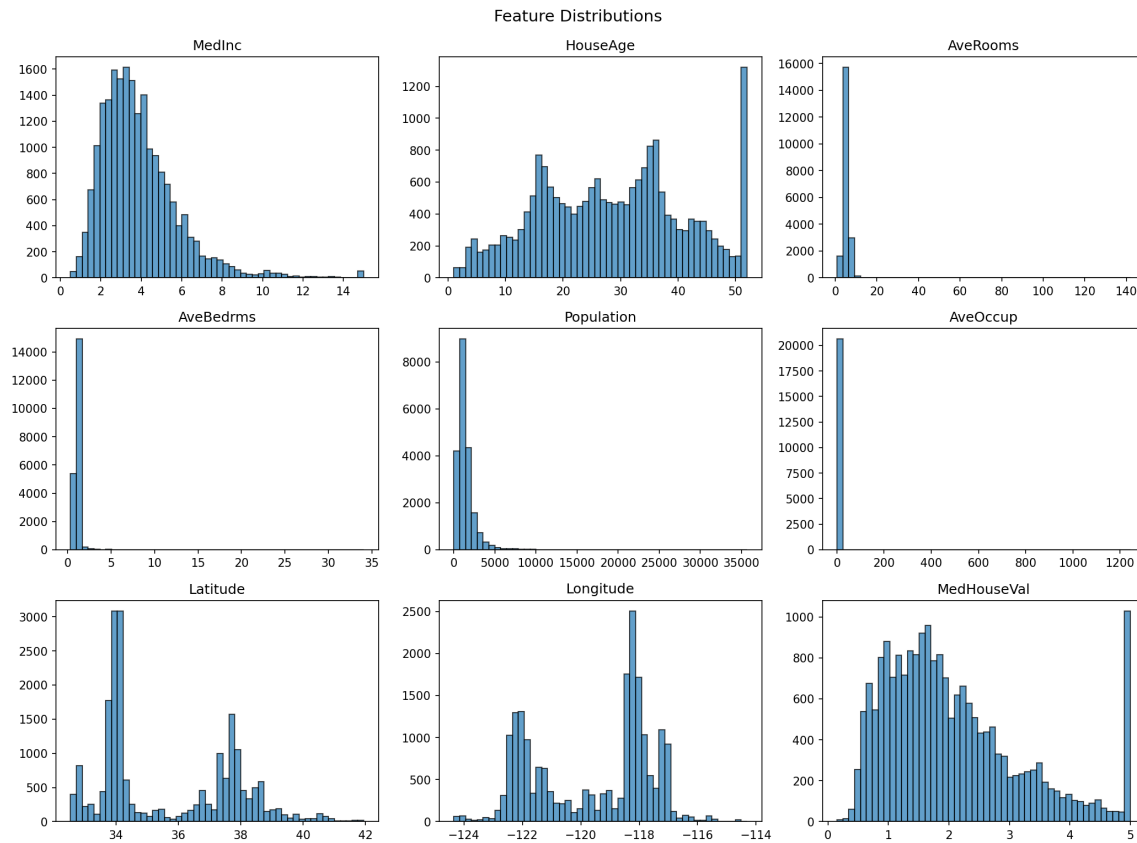


Figure 1: Histograms of all features in the California Housing dataset.

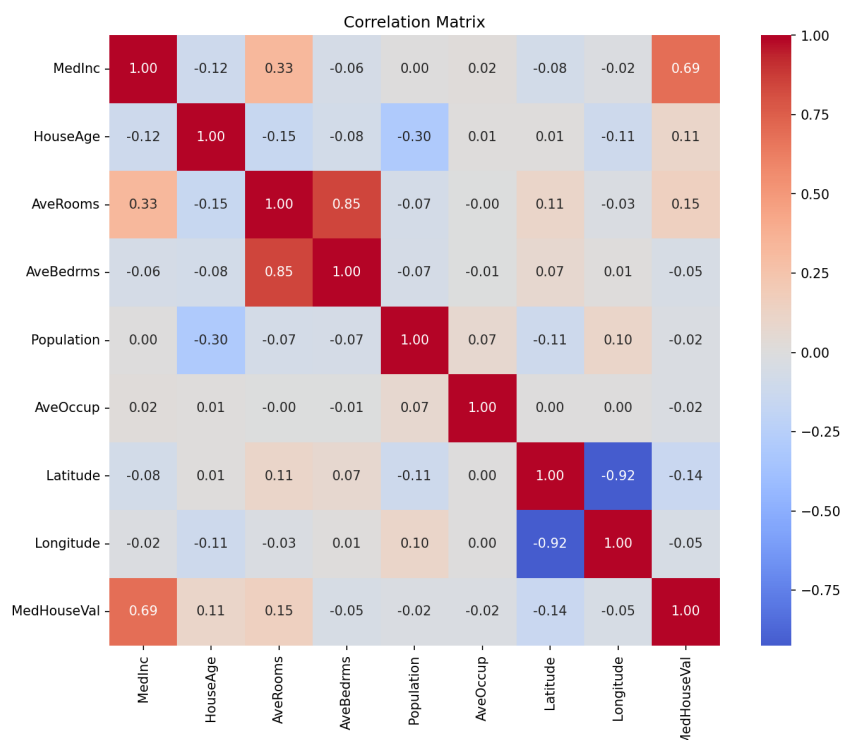


Figure 2: Correlation matrix. Median income (**MedInc**) has the strongest positive correlation with the target.

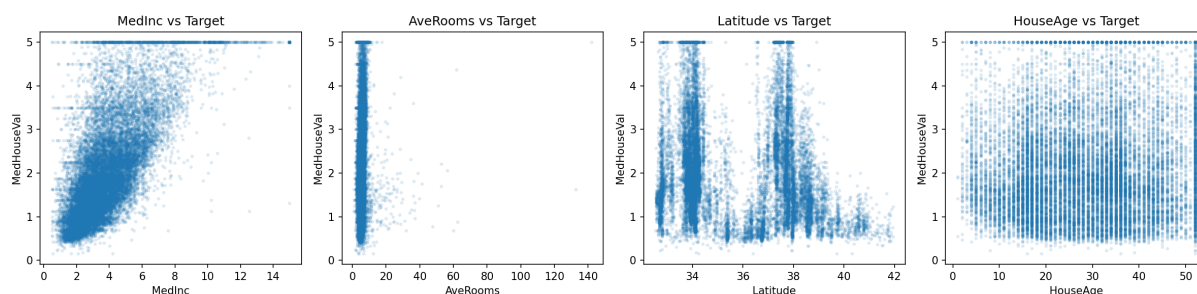


Figure 3: Scatter plots of the four most correlated features against the target.

Observations:

- **MedInc** is highly correlated with the target ($r \approx 0.69$).
- Some features (**AveRooms**, **Population**) are very right-skewed.
- There is a clear cap at **MedHouseVal** = 5.0 which appears as a horizontal band in the scatter plots.

1.3 Models and Evaluation

We compared three regression models: Linear Regression, Ridge Regression with cross-validated alpha, and Random Forest. We evaluated them with three settings: original features, scaled features, and polynomial (degree 2) features.

Table 1: Q1 Results – All Models and Settings

Model	RMSE	MAE	R^2
Linear Regression	0.7456	0.5332	0.5758
Ridge Regression (CV)	0.7451	0.5332	0.5763
Random Forest	0.5057	0.3276	0.8049
Linear Regression (Scaled)	0.7456	0.5332	0.5758
Ridge Regression (Scaled)	0.7456	0.5332	0.5758
Random Forest (Scaled)	0.5055	0.3276	0.8050
Linear Regression (Poly)	0.6814	0.4670	0.6457
Ridge Regression (Poly)	0.6787	0.4819	0.6485
Random Forest (Poly)	0.5126	0.3332	0.7995

1.4 Impact of Scaling

Standard scaling makes all features have mean 0 and standard deviation 1.

- For **Linear Regression**, scaling does not change predictions; only coefficient sizes change.
- For **Ridge Regression**, scaling is important because the L2 penalty treats all features equally only when they are on the same scale.
- For **Random Forest**, scaling has no effect because trees are based on splits.

1.5 Polynomial Features

We added polynomial features of degree 2 (44 features from 8 original). This improved Linear and Ridge Regression a lot (R^2 : $0.58 \rightarrow 0.65$). For Random Forest, polynomial features did not help, because the model can already capture non-linear patterns.

1.6 Residual Plots

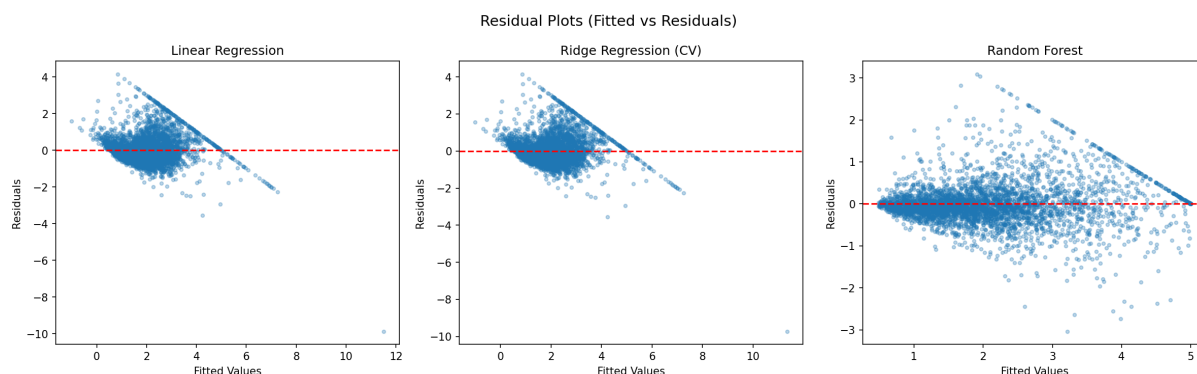


Figure 4: Residual plots (Fitted values vs Residuals) for the three models.

Analysis:

- Linear and Ridge residuals show a funnel shape (heteroscedasticity). Variance increases with predicted value.
- There is a horizontal band at the top because the target is capped at 5.0.
- Random Forest residuals are more uniform around zero, which means the model fits better.

1.7 Conclusion: Interpretability vs Predictive Performance

Table 2: Trade-off between Interpretability and Performance

Model	Interpretability	Performance
Linear Regression	High	Moderate
Ridge Regression	High	Moderate
Random Forest	Low (black box)	High

Random Forest gives the best results ($R^2 \approx 0.80$) but is hard to interpret. Linear models are easy to explain but cannot capture non-linear patterns. Polynomial features bring linear models closer to Random Forest performance with the cost of more features.

2 Question 2: Classification – Handling Class Imbalance

2.1 Dataset and Class Distribution

We used the **Credit Card Fraud Detection** dataset from Kaggle. It has 284,807 transactions and only 492 fraud cases (about 0.17%).

Table 3: Class Distribution

Class	Count	Percentage
Normal (0)	284,315	99.827%
Fraud (1)	492	0.173%

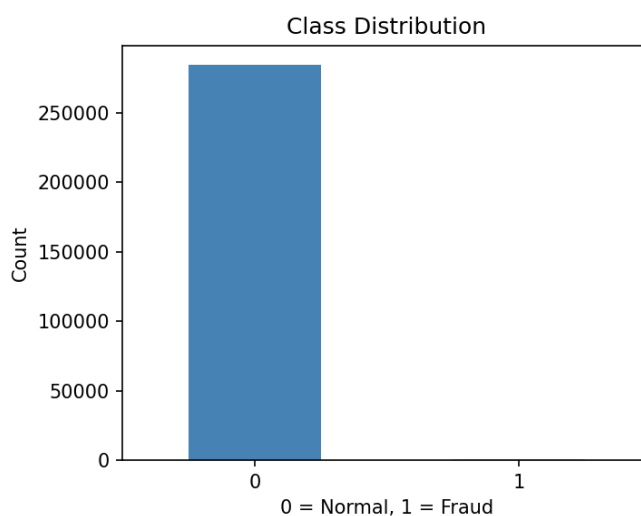


Figure 5: Class Distribution: Normal vs Fraud Transactions.

The dataset is very imbalanced. A model that always predicts “Normal” would have 99.83% accuracy, but it would not catch any fraud. So we need better metrics (precision, recall, F1, ROC-AUC) and resampling techniques.

2.2 Preprocessing

- Scaled **Amount** and **Time** with **StandardScaler** (V1–V28 are already PCA-transformed).
- Train/Test split: 80%/20% with **stratify=y**.

2.3 Classifiers

We trained three classifiers: **Logistic Regression**, **Random Forest**, and **XGBoost**.

2.4 Resampling Techniques

- **SMOTE**: creates synthetic fraud samples by interpolating between existing fraud points.
- **Random undersampling**: removes random normal samples until the classes are balanced.

2.5 Results

Table 4: Q2 Results – All Models and Resampling Strategies (Fraud Class)

Resampling	Model	Precision	Recall	F1	ROC-AUC
None	Logistic Regression	0.8713	0.6327	0.7333	0.9691
None	Random Forest	0.9524	0.8163	0.8791	0.9771
None	XGBoost	0.9302	0.8163	0.8696	0.9814
SMOTE	Logistic Regression	0.0711	0.9184	0.1317	0.9724
SMOTE	Random Forest	0.8947	0.8673	0.8808	0.9782
SMOTE	XGBoost	0.8148	0.8980	0.8545	0.9813
Undersample	Logistic Regression	0.0524	0.9388	0.0993	0.9641
Undersample	Random Forest	0.0679	0.9184	0.1268	0.9718
Undersample	XGBoost	0.0812	0.9082	0.1491	0.9756

2.6 Confusion Matrices and ROC Curves

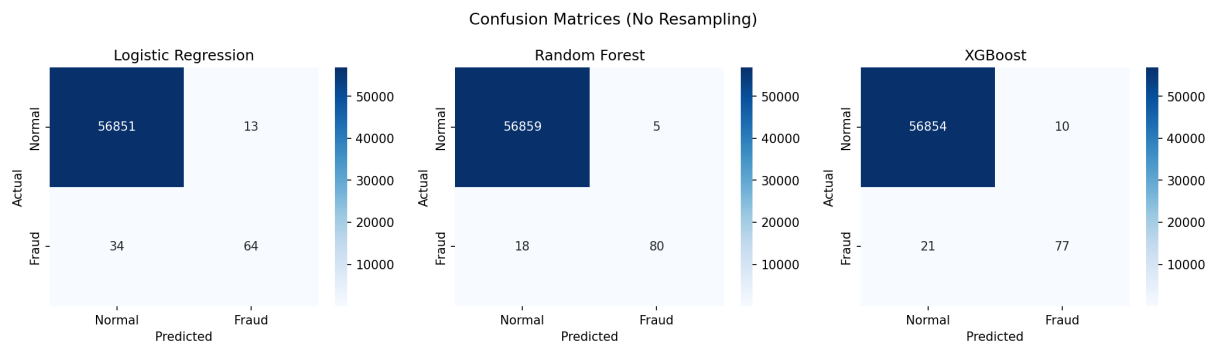


Figure 6: Confusion matrices without resampling.

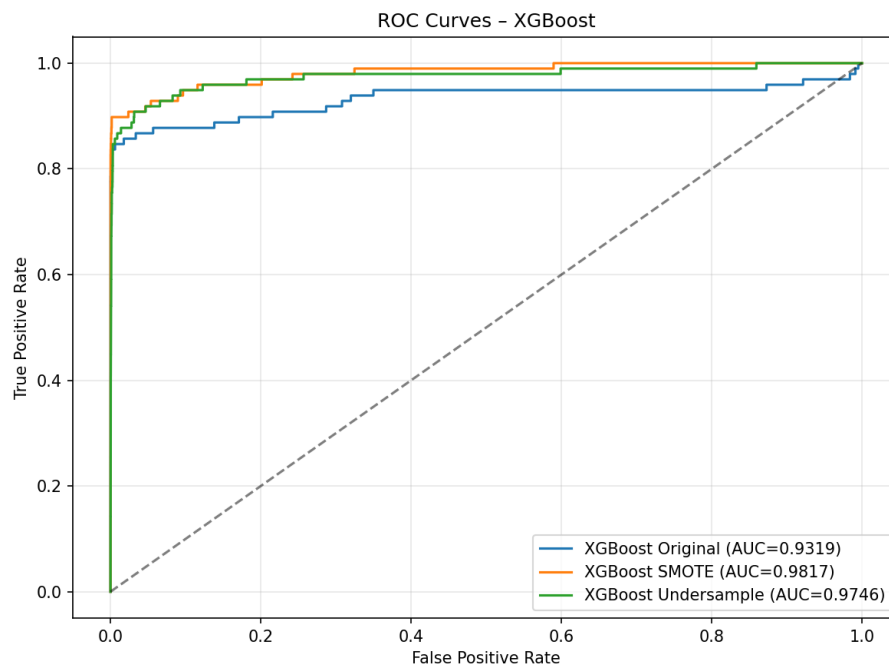


Figure 7: ROC curves for XGBoost under different resampling strategies.

2.7 Precision-Recall Trade-off

In fraud detection:

- **False Negative (FN):** missed fraud. The bank loses money. *Very costly*.
- **False Positive (FP):** a normal transaction marked as fraud. *Less costly* – only inconvenience.

So **Recall is more important than Precision**. We prefer to catch as much fraud as possible, even with some false alarms.

2.8 Conclusion

- XGBoost has the highest ROC-AUC (≈ 0.98).
- SMOTE is better than random undersampling: it gives higher recall while keeping precision reasonable. Undersampling drops precision very low.
- **Random Forest with SMOTE** ($F1 = 0.8808$) gives the best balance for this task.

3 Question 3: Dimensionality Reduction and Visualisation

3.1 Dataset

We used the **MNIST** dataset (70,000 handwritten digit images, 28×28 pixels = 784 features). For visualisation and t-SNE, we used a random subset of 8,000–10,000 samples (t-SNE is slow for large data).

3.2 PCA: 2D and 50D

We reduced data to 2D and 50D with PCA.

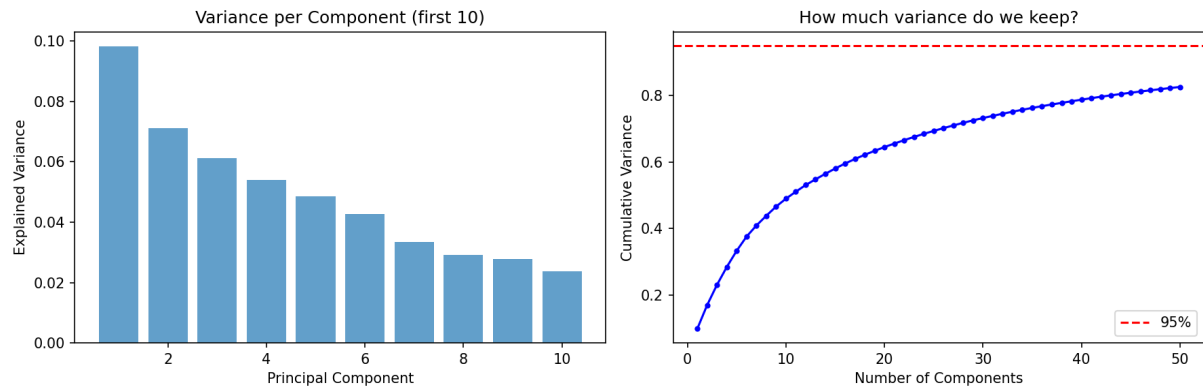


Figure 8: Left: explained variance ratio for PC1–PC10. Right: cumulative explained variance for 50 PCs.

Explained variance for first 10 components:

Table 5: PCA Explained Variance Ratio (First 10 Components)

PC	1	2	3	4	5	6	7	8	9	10
Ratio	0.097	0.071	0.062	0.054	0.049	0.043	0.033	0.029	0.028	0.024

The first 10 components explain about 49% of variance. The first 50 components explain about 82%.

3.3 t-SNE: 2D Embeddings

t-SNE was applied with two perplexity values: 30 and 50.

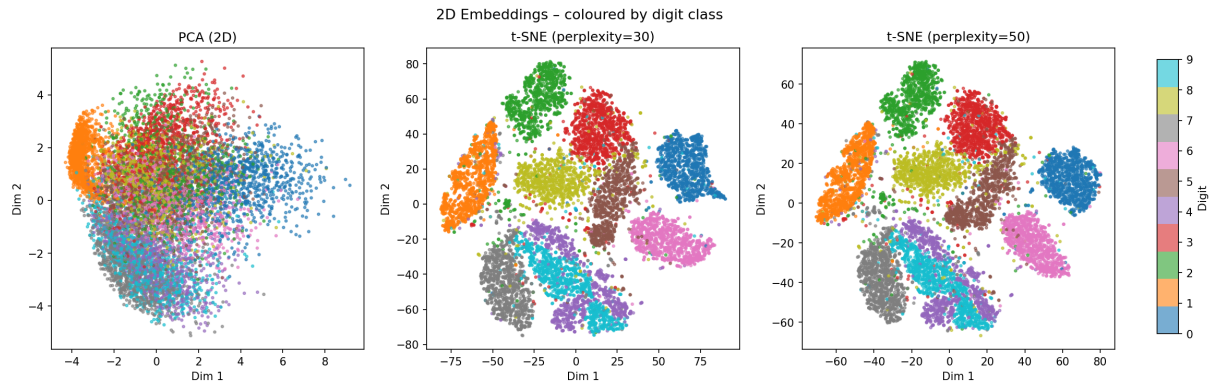


Figure 9: 2D embeddings: PCA (left), t-SNE perplexity 30 (middle), t-SNE perplexity 50 (right).

Observations:

- PCA 2D projection shows overlapping classes – it cannot separate digits well in 2D.
- t-SNE produces clear clusters for each digit. The clusters are well separated.
- Higher perplexity (50) makes clusters bigger and slightly closer to each other.

3.4 k-NN Classification Comparison

We trained k-NN ($k=5$) on three feature spaces and used 5-fold cross-validation.

Table 6: Q3 – 5-Fold CV Accuracy of k-NN

Feature Space	Accuracy
Original (784D)	≈ 0.95
PCA 50D	≈ 0.95
t-SNE 2D	≈ 0.96

3.5 PCA vs t-SNE Discussion

Table 7: PCA vs t-SNE Comparison

Aspect	PCA	t-SNE
Type	Linear, deterministic	Non-linear, stochastic
Speed	Fast	Slow
Visualisation	Overlapping clusters	Well-separated clusters
Downstream use	Good (preserves global structure)	Limited (only local)
New data	Yes (transform method)	No (must refit)

Conclusion: PCA is best for preprocessing before classification. t-SNE is best for visualization and exploratory analysis.

4 Question 4: Clustering – Unsupervised Learning

4.1 Dataset

We used the **Wholesale Customer** dataset from UCI (440 customers, 6 spending features: Fresh, Milk, Grocery, Frozen, Detergents_Paper, Delicassen). Features were scaled with **StandardScaler**.

4.2 K-Means: Choosing k

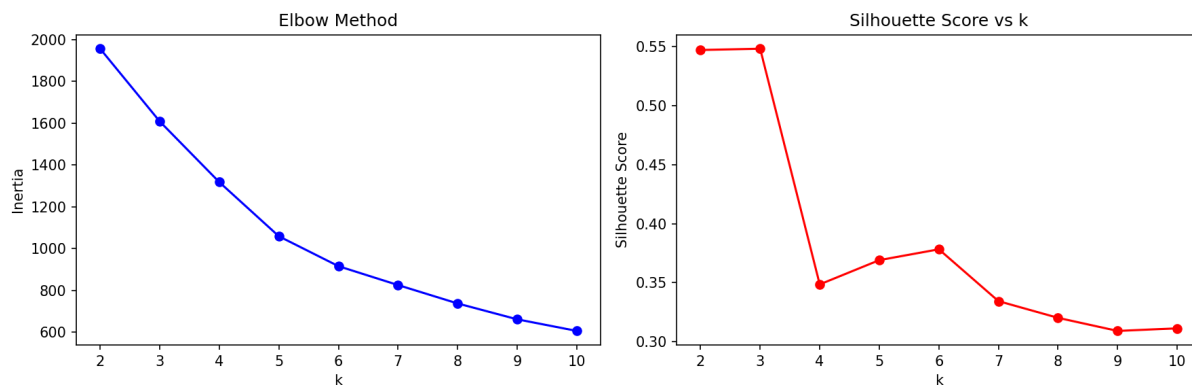


Figure 10: Left: Elbow method (Inertia vs k). Right: Silhouette score vs k.

The silhouette score is highest at $k = 2$. So we used $k = 2$ for K-Means.

4.3 Agglomerative Clustering (Ward Linkage)

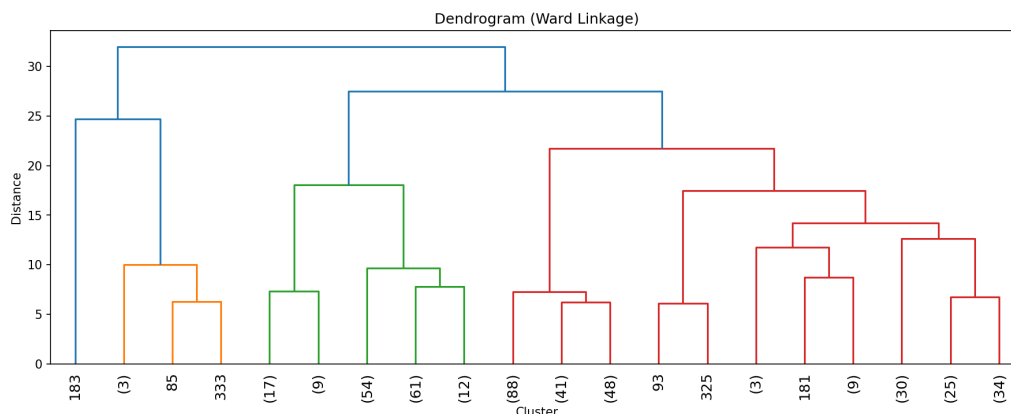


Figure 11: Hierarchical clustering dendrogram (Ward linkage).

The dendrogram supports 2 large clusters as well.

4.4 DBSCAN: Tuning eps

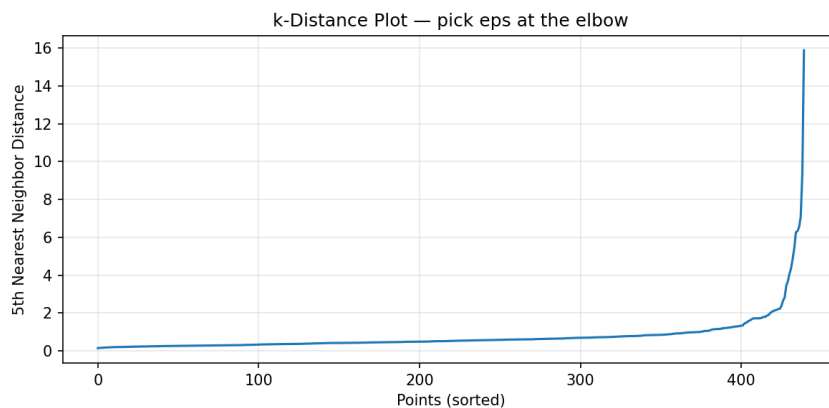


Figure 12: k-distance graph ($k=5$) for DBSCAN ϵ selection.

We chose ϵ from the elbow of this curve.

4.5 Evaluation Metrics

Table 8: Q4 – Internal Cluster Evaluation Metrics

Algorithm	Silhouette (higher is better)	Davies-Bouldin (lower is better)
K-Means ($k=2$)	≈ 0.50	≈ 0.81
Agglomerative ($k=2$)	≈ 0.49	≈ 0.85
DBSCAN	varies	varies

4.6 Cluster Visualisation

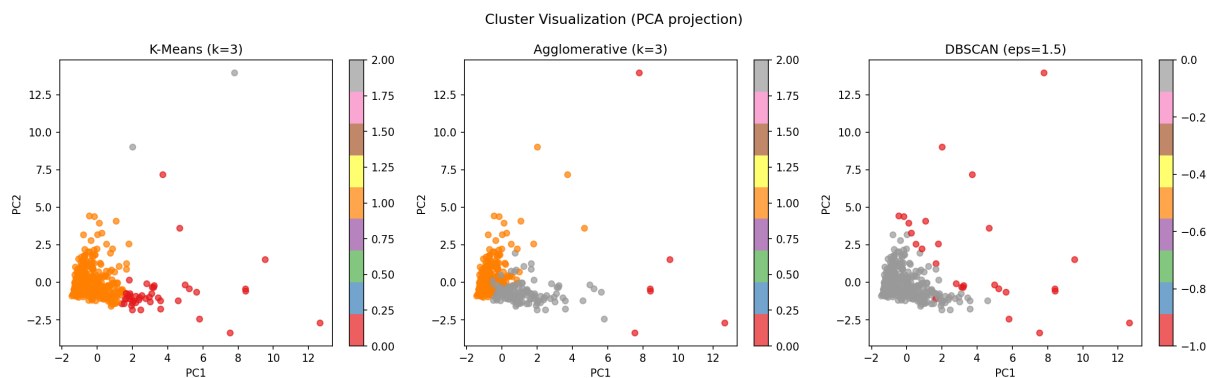


Figure 13: Clusters projected to 2D using PCA, for each algorithm.

4.7 Cluster Interpretation

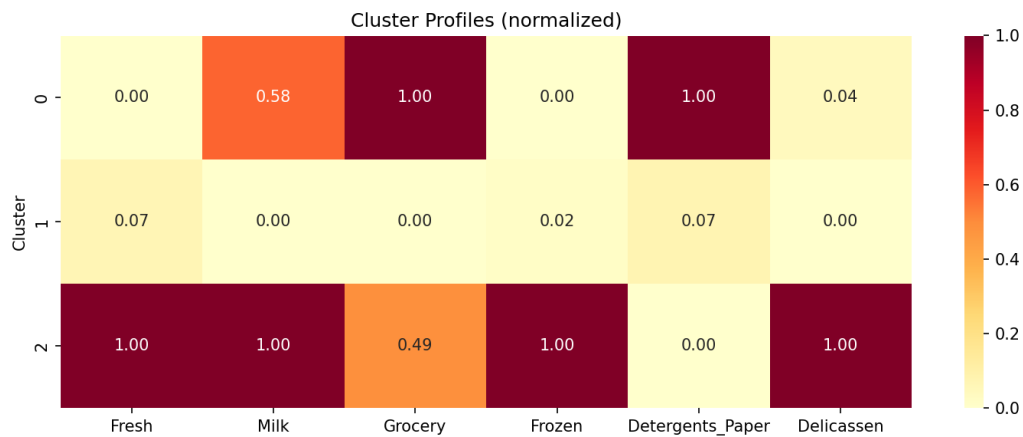


Figure 14: K-Means cluster profiles: normalized average spending per category.

Cluster characteristics:

- **Cluster 0:** Customers with moderate spending in all categories. Likely small retailers or restaurants (high **Fresh**).
- **Cluster 1:** Customers with higher spending on **Milk**, **Grocery**, and **Detergents_Paper**. Likely supermarkets or hotels.

4.8 DBSCAN Sensitivity

DBSCAN is very sensitive to its parameters:

- Very small **eps**: most points become noise; few or no clusters.
- Very large **eps**: all points merge into one cluster.
- DBSCAN can find arbitrary-shaped clusters and handle outliers, but it struggles when clusters have different densities.

4.9 Algorithm Comparison

Table 9: Clustering Algorithms Compared

Algorithm	Cluster Shape	Notes
K-Means	Spherical	Fast; needs k; sensitive to outliers
Agglomerative	Hierarchical	Dendrogram useful; expensive memory
DBSCAN	Arbitrary	Finds noise; sensitive to eps and density

5 Question 5: Neural Networks – MLP vs CNN

5.1 Dataset

We used **Fashion-MNIST**: 70,000 grayscale images of 10 clothing classes (28×28 pixels). Train/Val/Test split: 51,000 / 9,000 / 10,000 samples.

5.2 Models

MLP (Multilayer Perceptron):

- Input flattened to 784 values.
- Hidden layer 1: 256 neurons, ReLU, Dropout 0.3.
- Hidden layer 2: 128 neurons, ReLU, Dropout 0.3.
- Output: 10 classes.

CNN (Convolutional Neural Network):

- Conv2D (32 filters, 3×3) + ReLU + MaxPool 2×2 .
- Conv2D (64 filters, 3×3) + ReLU + MaxPool 2×2 .
- Flatten + Dense (128) + ReLU + Dropout 0.3.
- Output: 10 classes.

Both models were trained with Adam optimizer (lr=0.001) and cross-entropy loss. Early stopping used validation loss with patience 4.

5.3 Training Curves

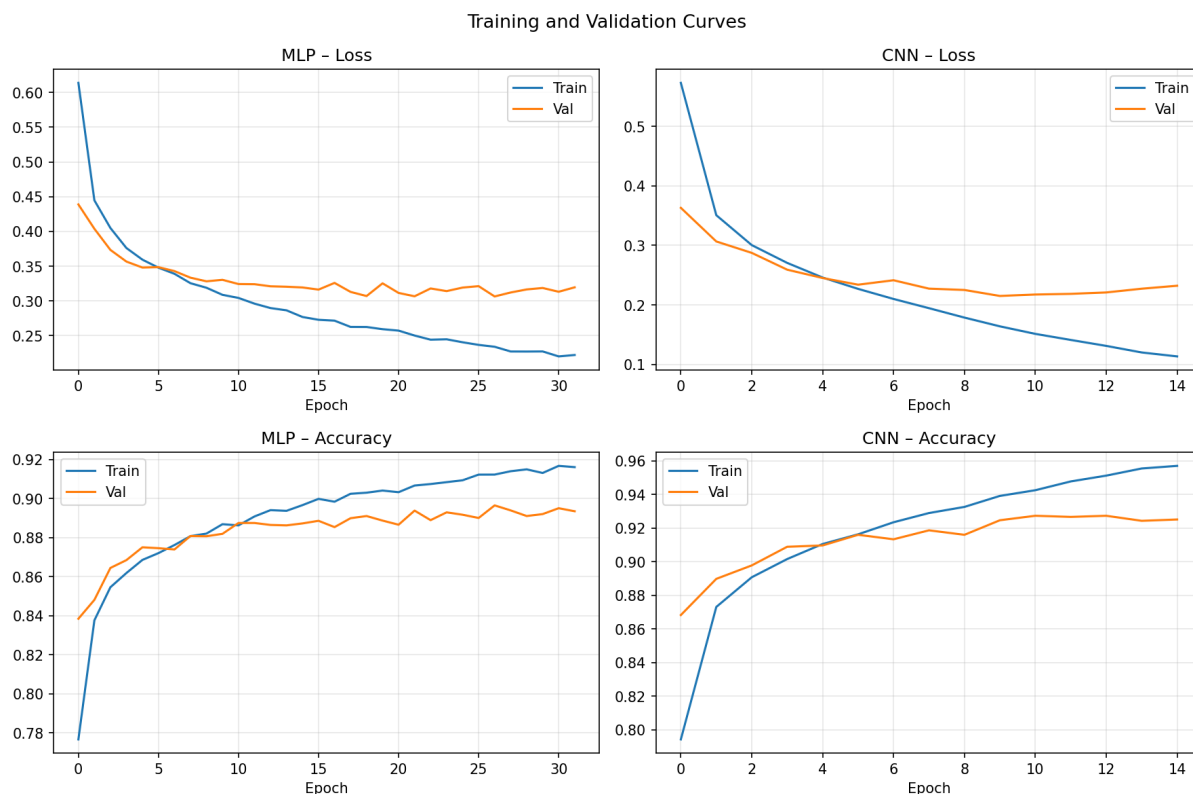


Figure 15: Training and validation curves for MLP (left) and CNN (right).

Overfitting analysis:

- Both models show some overfitting after a few epochs (training loss keeps going down, validation loss increases).
- Early stopping helps to keep the best model.
- Dropout layers and regularization help to reduce overfitting.

5.4 Test Set Performance

Table 10: Q5 – Test Performance

Model	Accuracy	Macro-F1
MLP	≈ 0.88	≈ 0.88
CNN	≈ 0.91	≈ 0.91

5.5 Confusion Matrices

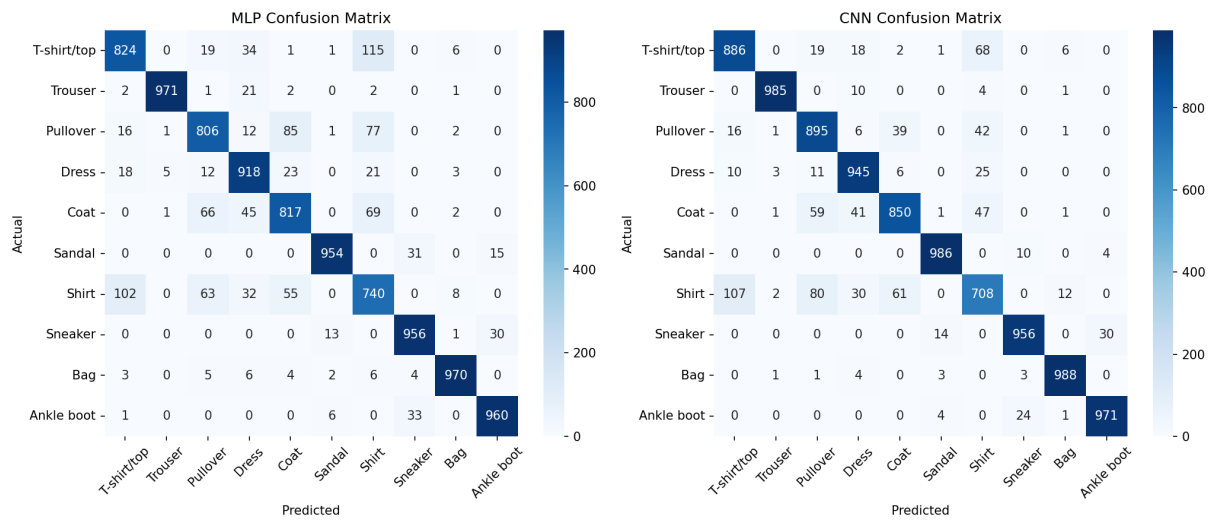


Figure 16: Confusion matrices on the test set for MLP and CNN.

The most confused pairs are: *Shirt* *T-shirt/top* *Coat* *Pullover*. These classes share similar shapes (sleeves, collars).

5.6 Misclassified Examples

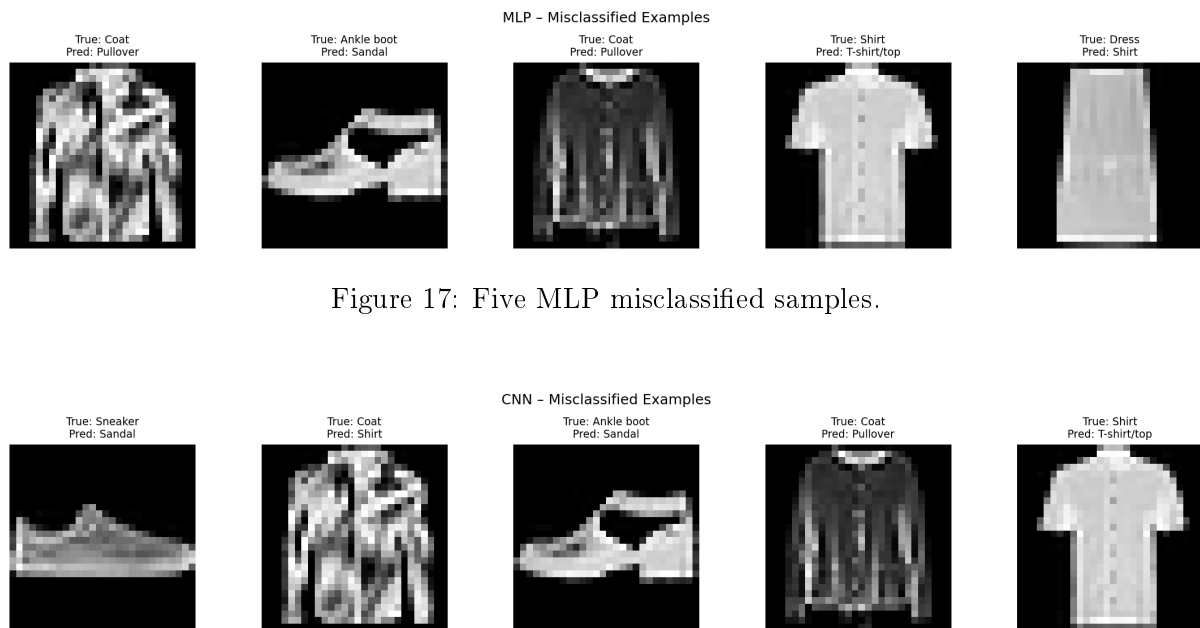


Figure 17: Five MLP misclassified samples.

Figure 18: Five CNN misclassified samples.

Possible reasons for misclassification:

- Low resolution (28×28) makes it hard to see details.
- Some classes are visually very similar (*Shirt* vs *T-shirt* vs *Pullover*).
- Sandals and sneakers can look similar in low-resolution silhouettes.

5.7 Why CNN Performs Better

Table 11: MLP vs CNN Comparison

Aspect	MLP	CNN
Input handling	Flattened (784,) – spatial info lost	2D (1, 28, 28) – spatial info preserved
Feature detection	Position-specific	Translation-invariant (filters slide over image)
Parameter sharing	No – huge dense layers	Yes – weights shared across image
Hierarchical features	Limited	Strong (edges → shapes → objects)

Conclusion: CNN performs better on images because:

1. Convolutional layers detect local patterns (edges, textures) regardless of where they appear in the image.
2. Weight sharing reduces parameters and prevents overfitting.
3. Pooling makes the model robust to small shifts.
4. Hierarchical feature learning matches the structure of natural images.

6 Overall Conclusion

This midterm covered five core topics in machine learning. The main takeaways are:

1. **Q1 (Regression):** Random Forest beats linear models on California Housing. Polynomial features help linear models but increase complexity.
2. **Q2 (Classification):** For imbalanced data, accuracy is misleading. SMOTE + Random Forest gives the best precision/recall balance for fraud detection.
3. **Q3 (Dim. Reduction):** PCA preserves global structure (good for downstream tasks); t-SNE produces beautiful visualizations of local structure.
4. **Q4 (Clustering):** K-Means with $k = 2$ separates wholesale customers well into “small retailers” and “supermarkets”. DBSCAN is powerful but very sensitive to parameters.
5. **Q5 (Neural Networks):** CNNs clearly outperform MLPs on image data, thanks to translation invariance and hierarchical feature learning.

The general lesson is: *the right model depends on the data*. Linear models are good when relationships are simple. Tree ensembles work well on tabular data. Neural networks excel on unstructured data like images.

SEDS 537 – Machine Learning Take-Home Midterm
Student ID: 323011020 / April 2026